Full Stack Engineer Take Home Assignment

As a Full Stack Engineer, you will be responsible for creating a seamless and intuitive interface for our internal teams. While you are expected to maintain a functional backend to support the features, this assignment is your opportunity to showcase your expertise in State Management, Component Architecture, and responsive UI/UX that handles data-rich environments effectively.

1. Background

Your task is to design and implement an internal dashboard to monitor incoming payments. To help you get started, we have provided a **backend boilerplate** that includes basic authentication logic, database migrations, and a predefined OpenAPI. While the API structure for payment endpoints is already defined, the implementation logic is left for you to complete.

Your main challenge is to implement the missing backend logic and integrate it into a fully functional frontend dashboard.

2. Requirements

2.1. Functional Requirements (Backend Provided)

The following endpoints are already available in the boilerplate, but you may need to ensure they are correctly integrated and extended if necessary:

2.1.1. Authentication

Endpoint: `/dashboard/v1/auth/login`

- Accepts **email** and **password**.
- Returns a valid **JWT token** and user **role** (`cs` or `operation`).

- Used by the frontend to determine access rights.

2.1.2. List Payments

Endpoint: `/dashboard/v1/payments`

- Protected endpoint (authentication required).
- Returns the full list of payments.

2.1.3. Filter Payments by Status

Endpoint: `/dashboard/v1/payments?status=<status>`

- Protected endpoint (authentication required).
- Returns only payments that match the given **status** (`completed`, `processing`, or `failed`).

2.2. Frontend Requirements

> ✅ **Design Freedom:** You're free to design the layout/UI, page structure, styling, and color palette however you like. You may use any component library (or none). Creativity is encouraged as long as all required features are met

2.2.1. Login Page

- Simple form with **email** and **password** fields.
- On success, store **JWT token** and **role** in client state.
- Redirect user to the Dashboard page.

2.2.2. Dashboard Page (Protected Route)

- Accessible only after successful login.
- Fetches and displays payments from the API.
- **Table columns**: Payment ID, Merchant Name, Date, Amount, Status.

- **Summary widget**: Show total payments and breakdown (e.g., `Total: 50 | Success: 40 | Failed: 10`).

2.2. Non-Functional Requirements

> 💡 We expect both backend and frontend to be runnable. Since this role leans towards frontend expertise, prioritize a polished UI and clean state management.

2.2.1. Tech Stack & Libraries

- **Backend:** Golang (using the provided boilerplate).
- **Frontend:** Vue.js or React. (JS/TS)
- **Persistence:** In memory, Redis or SQLite.
- **API Style:** RESTful, defined using **OpenAPI v3** (`openapi.yaml`).
- **Validation:** Use JSON Schema or Go structs for validating requests/responses.
- **Production Readiness:** Design your implementation to be effective in real-world, multi-user environments

2.2.2 Project Setup

- See boilerplate from https://github.com/durianpay/fullstack-boilerplate
- Include `README.md`, `Makefile`.
- API spec (`openapi.yaml`) at repo root.
- Clear setup instructions for running backend and frontend locally (e.g., via `make` or `docker-compose`).

2.2.3 Testing & Code Quality

- Backend: Basic unit tests for critical logic.
- Frontend: Component modularity and clean state handling.

- Follow standard linting and formatting for both languages.

3. Submission Guidelines & Review Mechanism

3.1. Deliverables

Provide a working service with clear setup instructions.

- Submissions must be in a **single repository** with two directories:
  - `backend`
  - `frontend`
- The repository must include a **clear root** `README.md` that provides:
  - All prerequisites and dependencies
  - Step-by-step environment setup instructions for both backend and frontend
  - Providing data seed
  - Exact commands to build, start, and test the project
  - API documentation (preferably in **OpenAPI/Swagger** format).

Frontend (Vue/React):

- Login page, protected dashboard, payment table, summary widget
- Clear environment/config instructions (e.g., API base URL).
- Instructions to run dev server and build for production.
- Optional: component tests for critical views.

Reminder:
This assignment should be completed **within 72 hours** of receiving it by email.

3.2. How to Submit

Upload your code to a new GitHub repository (public or private).

If your repository is private, add the GitHub account `durianpay-tech-interviewer` as a collaborator so we can review your work.

(If you have difficulties adding the above GitHub account, you can add `arifbudurian` or `fajrinajiseno2` instead)

No starter UI template or project skeleton is provided; structure the project as you think is best.

3.3. Reviewer Environment

Your submission will be reviewed on a machine with:

```
Go v1.21+
Node v20+
Docker & Docker Compose
Makefile
Mac-based computer
```

Please ensure your code and instructions are compatible with this environment.

3.4. Validating The Test

Submissions that fail to execute properly following your instructions in README.md our reviewer environment will be **automatically rejected**. We recommend testing your setup instructions on a clean environment before submission.

3.5. Post-Submission Process

After you submit your assignment, our technical team will review your code within 2 to 5 business days.

You will receive an email with feedback and next steps regardless of the outcome.

For any technical questions during the assignment, please contact tech-recruitment@durianpay.id

4. Evaluation Criteria

Your code will be evaluated based on several key factors:

- **UI/UX Implementation:** How well you translate requirements into a functional interface.
- **Frontend Logic & State:** Clean management of application state.
- **Component Quality:** Modular, maintainable, and well-structured frontend code.
- **API Integration:** Correct handling of API calls and HTTP status codes.
- **Functional Reliability:** All features must work end-to-end.
- **Unit test**: adding unit test for backend and frontend, explain your testing strategies in README.md
- **Openapi integration:** FE need to integrate with backend with openapi integration
- **Basic Backend Implementation:** A clean, working Go backend that serves as a reliable data source for your frontend application.

Remember, this assignment is not just about making the features work, but about how you organize your frontend components and communicate your technical decisions. We encourage you to show your creativity in building a clean, user-friendly interface while meeting all the functional requirements.

We look forward to seeing your submission. Good luck!